CiT Backend Engineering Track

Week 15: OAuth2, OIDC & Keycloak

Delegated Identity, Single Sign-On & Securing the Library API

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 2 — Specialisation (Weeks 9–20)
Week	Week 15
Instructor	Jim & Anderson
Duration	6 months | 24 weeks
Tech Stack	Java 25, Spring Boot 4, Spring Security, Keycloak, PostgreSQL, Postman

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we finish a module of our training, and we thank You for every lesson learned along the way. We learn to rely on what is trustworthy, to handle what is entrusted to us with care, and to prove our work is sound before we hand it to others. Make us faithful in the small things, so that we can be trusted with greater ones.

As Matthew 25:21 records, "Well done, good and faithful servant; you were faithful over a few things, I will make you ruler over many things." Lord, let us be faithful in the work before us today, and let that faithfulness prepare us for all You have ahead.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,
Amen.

Scripture for the Week

"Well done, good and faithful servant; you were faithful over a few things, I will make you ruler over many things."

— Matthew 25:21 (NKJV)

Table of Contents

1. Welcome to Week 15
2. The Problem OAuth2 Solves
3. OAuth2 — Roles & the Authorization Code Flow
4. OpenID Connect — Adding Identity
5. Keycloak — An Identity Platform
6. Spring Boot as an OAuth2 Resource Server
7. Mapping Keycloak Roles to Spring Security
8. Module Project — Secure the Library API
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 15

In Week 13 you built authentication yourself: a login endpoint that checked a password and issued your own signed JWT. In Week 14 you learned to defend an API against the most common attacks. This week completes the API Design & Security module with the way real companies actually handle identity.

In professional systems you rarely write your own login. Storing passwords, resetting them, locking accounts, adding two-factor codes, and signing tokens safely is hard to get right — so organisations hand that job to a dedicated identity platform. The open standards that make this possible are OAuth2 and OpenID Connect (OIDC), and the platform used across the XENO stack is Keycloak.

By the end of today your API will stop issuing tokens altogether. Keycloak will log users in and issue the tokens; your Spring Boot API will simply check them. You will then finish the module project: securing the Library API with two distinct user roles.

What to Expect in Week 15

By the end of this session, you will be able to:

- Explain why applications delegate login to a central identity platform
- Name the four OAuth2 roles and walk through the Authorization Code flow with PKCE
- Explain what OpenID Connect adds, and the difference between an ID token and an access token
- Describe Keycloak's core concepts — realms, clients, users, and roles
- Configure Spring Boot as an OAuth2 resource server that trusts Keycloak's tokens
- Map Keycloak roles into Spring Security and protect endpoints by role
- Complete the module project: a Library API secured with two roles

2. The Problem OAuth2 Solves

Before learning a new standard, understand the pain it removes. Two problems led the industry to OAuth2.

2.1 Every App Building Its Own Login

Imagine an organisation with five applications — a student portal, a library system, a fees system, and two more — each with its own login. Every one of them stores passwords, so there are five databases an attacker can target. Users juggle five passwords and reuse them. And five separate teams must each get hashing, account lockout, password reset, and two-factor codes exactly right. One weak implementation puts everyone at risk.

Every app builds its own login	One central identity platform
Passwords stored in many places	Passwords stored in exactly one place
A separate account for each app	One login for every app (single sign-on)
Every team re-builds reset, lockout, two-factor	Built once, by a dedicated, well-tested platform
Security quality varies app by app	One place to audit, strengthen, and monitor

2.2 The Password-Sharing Trap

The second problem is sharing access. Suppose a budgeting app wants to read your mobile money transactions. The naive solution is to hand it your PIN. But now that app can do anything you can — including send money — for as long as it likes, and the only way to stop it is to change your PIN everywhere.

What you really want is to grant limited access: "this app may read my transactions, nothing more, and I can switch that off at any time." That idea — delegated, limited, revocable access — is exactly what OAuth2 provides.

The Hotel Key Card

At a hotel in Kampala, you show your ID once at reception. Reception gives you a key card that opens only your room, only for the nights you booked. The room door

never sees your ID — it simply checks the card. Reception is the identity platform, the key card is the token, and the door is your API. If the card is lost, reception cancels it without changing anything else.

3. OAuth2 — Roles & the Authorization Code Flow

OAuth2 is an open standard for granting access with tokens instead of passwords. It defines who is involved and the exact steps they follow. Learn the four roles first; the flow then makes sense.

3.1 The Four Roles

Role	Plain English	In Our System
Resource Owner	The person whose data it is	A library member
Client	The app that wants access on their behalf	The Angular library portal
Authorization Server	Logs the user in and issues tokens	Keycloak
Resource Server	Holds the protected data; checks tokens	Your Spring Boot Library API

3.2 The Authorization Code Flow

This is the flow used by web and mobile apps. Follow it step by step:

1. The member clicks "Log in" in the portal. The portal redirects the browser to Keycloak's login page.
2. The member types their username and password into Keycloak — never into the portal itself.
3. Keycloak redirects back to the portal with a short-lived, single-use authorization code.
4. The portal exchanges that code at Keycloak's token endpoint for an access token (and an ID token).
5. The portal calls your API with the header Authorization: Bearer <access token> — exactly as in Week 14.
6. Your API checks the token's signature, expiry, and issuer, then returns the data.

The Client Never Sees the Password

This is the whole point. The portal, and your API, never touch the member's password — only Keycloak does. Older tutorials show a "password grant", where

the app collects the password and sends it to the identity server. It defeats the purpose of OAuth2 and is deprecated. Do not use it for real applications.

3.3 PKCE and Scopes

A browser or mobile app cannot keep a secret — anyone can read its code. PKCE (Proof Key for Code Exchange, pronounced "pixie") protects the flow anyway: the app creates a one-time random secret, sends a scrambled version with the login request, and proves it knows the original when exchanging the code. A stolen authorization code is useless without it. Authorization Code with PKCE is the modern standard for every browser and mobile client.

A scope names what access the client is asking for — for example `openid`, `profile`, or `email`. The user and the identity platform decide which scopes are granted.

Exercise 1: Trace the Flow

On paper, draw the six steps of the Authorization Code flow for the CiT Library portal. Label each arrow with who sends it, who receives it, and what it carries (a redirect, a code, a token, or data). Then mark the one place the password travels.

4. OpenID Connect — Adding Identity

Here is a subtle but important point: OAuth2 on its own is about access, not identity. An access token says "the bearer may read library data" — it was never designed to tell the portal who just logged in. OpenID Connect (OIDC) is a thin layer on top of OAuth2 that adds exactly that.

4.1 What OIDC Adds

Addition	What It Gives You
The openid scope	Asking for it turns an OAuth2 login into an OIDC login
The ID token	A JWT for the client that says who logged in and when
Standard claims	Agreed names such as sub, name, email, preferred_username
The userinfo endpoint	A URL the client can call for more profile details
Discovery	A well-known URL that describes every endpoint and key

4.2 ID Token vs Access Token

A login now returns two tokens, and each has a different job:

ID Token	Access Token
For the client (the portal)	For the resource server (your API)
Answers "who is this user?"	Answers "what may the bearer do?"
Used to show "Welcome, Alice"	Sent as Authorization: Bearer on every API call
Never sent to the API	Never used to identify the user in the client

Authentication and Authorisation, Standardised

Remember the distinction from Week 13? OIDC is the standard for authentication — proving who someone is. OAuth2 is the standard for authorisation — granting access. Keycloak speaks both, which is why one login can tell the portal who you are and tell the API what you may do.

4.3 Discovery — Configuration in One URL

Every OIDC provider publishes a discovery document. For a Keycloak realm named cit, it lives at:

```
http://localhost:8180/realms/cit/.well-known/openid-configuration
```

It lists the login URL, the token URL, and the location of the public signing keys. This is why, as you will see in Section 6, your Spring configuration needs only a single line — Spring reads everything else from this document.

5. Keycloak — An Identity Platform

Keycloak is a free, open-source identity and access management platform. It provides login pages, user management, password reset, two-factor authentication, single sign-on, and fully standard OAuth2 and OIDC — so your applications do not have to. It is the identity platform in the XENO stack.

5.1 Core Concepts

Concept	Plain English
Realm	An isolated space with its own users, apps, and settings — for example, a cit realm
Client	An application registered with Keycloak — the portal, the API, a mobile app
User	A person who can log in
Role	A named permission such as MEMBER or LIBRARIAN, assigned to users
Group	A set of users who share the same roles — assign once, applies to all

5.2 Running Keycloak Locally

Keycloak runs as its own server. The quickest way to start it is with Docker. You will study Docker properly in Week 18; for now, run this one command with your instructor's help. It uses port 8180 so it does not clash with your API on 8080.

```
docker run -p 8180:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin \
  -e KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak start-dev
```

Open http://localhost:8180 and sign in to the admin console. The start-dev mode and the admin/admin login are for learning on your own machine only — never for a real deployment.

5.3 Setting Up the Library Realm

7. Create a realm named cit

8. Create a client named library-portal — a public client with Standard Flow enabled and the redirect URI http://localhost:4200/*
9. Create two realm roles: MEMBER and LIBRARIAN
10. Create two test users, set their passwords, and give one MEMBER and the other LIBRARIAN

Single Sign-On

Once every CiT application is a client in the same realm, a user logs in once and is recognised by all of them — the portal, the library, the fees system. That is single sign-on (SSO). Log out once, and every app knows. One identity, one password, one place to secure.

Exercise 2: Build Your Realm

Complete the four set-up steps above in your own Keycloak. Then open the realm's discovery URL in your browser and find three things in the JSON: the token endpoint, the authorization endpoint, and jwks_uri (where the public keys live).

6. Spring Boot as an OAuth2 Resource Server

Now the big change to your API. In Week 13 it did two jobs: issuing tokens at /auth/login and checking them on every request. From today, Keycloak issues the tokens. Your API does only the second job — it becomes a resource server.

6.1 Add the Dependency and One Setting

Add the spring-boot-starter-oauth2-resource-server dependency, then tell Spring which issuer to trust:

```
# application.yml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8180/realms/cit
```

6.2 Update the Filter Chain

Replace your hand-written JWT filter from Week 13 with Spring's built-in resource-server support:

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/actuator/health").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(oauth2 -> oauth2
            .jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())));
    return http.build();
}
```

Your /auth/login endpoint, your token-generating code, and your signing secret can all be deleted. (The jwtAuthenticationConverter() method is written in Section 7.)

6.3 How Validation Works Now

In Week 13 your API signed and checked tokens with one shared secret. Keycloak uses a key pair instead: a private key, kept only inside Keycloak, signs each token; a public key, published at the realm's jwks_uri, lets anyone check the signature but cannot create one. On start-up Spring reads the discovery document, downloads the public keys, and then verifies every token's signature, expiry, and issuer.

Week 13 — your own tokens	Week 15 — Keycloak tokens
Your API issues tokens at /auth/login	Keycloak issues tokens after its own login page
One shared secret signs and verifies	Keycloak's private key signs; its public key verifies
The API must guard the signing secret	The API holds no secret at all
The API stores and checks passwords	Only Keycloak ever sees a password

Less Code, Less Risk

Notice what happened: your API became simpler and safer at the same time. There is no password table to leak, no signing key to guard, no login endpoint to brute-force. Each piece of your system now does one job well — the same principle behind the layered architecture you have used since Week 8.

7. Mapping Keycloak Roles to Spring Security

Your API now accepts Keycloak tokens — but it does not yet understand the roles inside them. Keycloak and Spring store roles in different places, so we add a small translator.

7.1 Where Keycloak Puts Roles

Decode a Keycloak access token and you will find the realm roles under a claim called realm_access:

```
{
  "iss": "http://localhost:8180/realms/cit",
  "sub": "f3a1c9e2-7b4d-4c1a-9e55-2d8b6a0c1f77",
  "preferred_username": "alice",
  "realm_access": { "roles": ["MEMBER"] },
  "exp": 1790000000
}
```

By default Spring only reads the scope claim, so it would not see MEMBER or LIBRARIAN at all. We teach it where to look.

7.2 A Role Converter

```
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(jwt -> {
        Map<String, Object> realmAccess = jwt.getClaim("realm_access");
        if (realmAccess == null) return List.of();
        List<String> roles = (List<String>) realmAccess.get("roles");
        return roles.stream()
            .map(role -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + role))
            .toList();
    });
    return converter;
}
```

Each Keycloak role becomes a Spring authority with the ROLE_ prefix, so everything you learned in Week 13 now works unchanged.

7.3 Protecting Endpoints by Role

Add @EnableMethodSecurity to your security configuration, then guard methods exactly as before:

```
@PreAuthorize("hasRole('LIBRARIAN')")
@PostMapping
public ResponseEntity<ApiResponse<BookResponse>> addBook(
        @Valid @RequestBody BookRequest request) { ... }
```

7.4 Who Is Calling? — Ownership Checks

Roles answer "what kind of user is this?" — not "is this their record?". The IDOR lesson from Week 14 still applies. Read the caller's identity from the verified token and check ownership in your service:

```
@GetMapping("/my-loans")
public ApiResponse<List<LoanResponse>> myLoans(@AuthenticationPrincipal Jwt jwt) {
    String memberId = jwt.getSubject();   // who is calling — from the token
    return new ApiResponse<>("SUCCESS", "Loans fetched",
                        loanFacade.findLoansFor(memberId));
}
```

Trust Only the Verified Token

Roles and the user id are trustworthy only because Spring has verified the token's signature against Keycloak's public key. Never accept a role, a user id, or an "isAdmin" flag from the request body, a query parameter, or a custom header — an attacker controls all of those. Identity comes from the token, and nowhere else.

Exercise 3: Roles End to End

Add the role converter and @EnableMethodSecurity to your API. Protect one endpoint with hasRole('LIBRARIAN'). Using tokens for your two test users, confirm that the member receives 403 Forbidden and the librarian succeeds.

8. Module Project — Secure the Library API

This project completes the API Design & Security module. You will secure the Library Management System you built in Week 12, applying everything from Weeks 13 to 15 — this time with Keycloak as the identity platform.

8.1 Requirements

The Secured Library API Must Have

- A Keycloak realm with two roles — MEMBER and LIBRARIAN — and at least one test user for each
- The API configured as an OAuth2 resource server trusting the realm's issuer
- Every endpoint requiring a valid token, except the Actuator health check
- Role rules enforced with @PreAuthorize, following the table below
- An ownership check so a member can only ever see and return their own loans
- 401 for a missing or invalid token and 403 for the wrong role — both in the standard envelope
- No secrets in the codebase, and the Week 14 audit fixes still in place
- The OpenAPI spec updated with a security scheme, and a Postman collection that obtains tokens

Endpoint	MEMBER	LIBRARIAN
GET /books (browse, paginated)	Allowed	Allowed
POST, PUT, DELETE /books	Forbidden (403)	Allowed
POST /loans (borrow a book)	Allowed — for themselves only	Allowed
GET /my-loans	Allowed — own loans only	Allowed
GET /loans (all members)	Forbidden (403)	Allowed

8.2 Documenting Security in OpenAPI

Tell every consumer of your API how to authenticate by adding a security scheme to docs/openapi.yaml (Week 8). The frontend team reads this to know exactly how to log in:

```
components:
  securitySchemes:
    keycloak:
      type: openIdConnect
      openIdConnectUrl: http://localhost:8180/realms/cit/.well-known/openid-configuration
security:
  - keycloak: []
```

8.3 Getting a Token in Postman

11. In your collection, open the Authorization tab and choose OAuth 2.0
12. Set Grant Type to "Authorization Code (With PKCE)"
13. Enter the Auth URL and Access Token URL from your realm's discovery document, and Client ID library-portal
14. Click "Get New Access Token", log in through Keycloak's page, and use the token
15. Repeat for both users and prove every row of the table above

Module Complete — API Design & Security

In three weeks you have learned authentication and authorisation, JWTs and Spring Security, defences against the OWASP Top 10, and delegated identity with OAuth2, OIDC, and Keycloak. That combination — a clean API that is genuinely secure — is precisely what separates a junior developer from a strong hire. Commit this project with care; it is a centrepiece of your portfolio.

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Why organisations delegate login to a central identity platform instead of every app building its own
- The four OAuth2 roles, and the Authorization Code flow with PKCE — the client never sees the password
- OpenID Connect adds identity: the ID token for the client, the access token for the API
- Keycloak's realms, clients, users, and roles — and single sign-on
- Spring Boot as an OAuth2 resource server that verifies tokens with Keycloak's public keys
- Mapping realm_access roles to Spring authorities, @PreAuthorize, and ownership checks from the token
- Completing the API Design & Security module with a fully secured Library API

9.2 Homework

16. Complete the module project — the Library API secured with Keycloak and two roles — and push it to GitHub
17. Update docs/openapi.yaml with the security scheme and commit a Postman collection that obtains tokens for both roles
18. In your own words, write a short README section explaining the difference between the ID token and the access token
19. Preview Week 16: Read about why systems use asynchronous messaging, and what Apache Kafka is. Come with one question ready.

Coming Next — Week 16: Messaging & Event-Driven Architecture

The Messaging & Event-Driven Architecture module begins. Not everything should happen inside a single request: sending a notification when a book is borrowed, for example, can happen in the background. You will learn why systems decouple work with asynchronous messages, and the core concepts of Apache Kafka — the event-streaming backbone of the XENO platform: topics, partitions, producers, consumers, and consumer groups.

Closing Reflection

"I am the good shepherd; and I know My sheep, and am known by My own."

— John 10:14 (NKJV)

This week was about identity — knowing, with certainty, who someone is before trusting them. Our systems need a trusted authority to vouch for each user. Our lives rest on something deeper: we are fully known by the Good Shepherd, and still loved. Build systems that treat every identity with that same care.

Glossary of Terms

Term	Definition
Access Token	A token sent to the API to prove what the bearer may do
Authorization Code	A short-lived, single-use code the client exchanges for tokens
Authorization Server	The service that logs users in and issues tokens — here, Keycloak
Client	An application registered with the identity platform, such as the portal
Discovery Document	The .well-known URL describing an OIDC provider's endpoints and keys
ID Token	A token for the client describing who logged in; never sent to the API
Issuer	The identity platform that created and signed a token (the iss claim)
JWKS	The published set of public keys used to verify token signatures
Keycloak	An open-source identity and access management platform used in the XENO stack
OAuth2	An open standard for granting limited access with tokens instead of passwords
OpenID Connect	An identity layer on top of OAuth2 that adds the ID token and standard claims
PKCE	Proof Key for Code Exchange — protects the code flow for browser and mobile apps
Realm	An isolated Keycloak space with its own users, clients, and roles
Resource Server	The API holding protected data, which verifies incoming tokens
Scope	A named piece of access a client requests, such as openid or email
Single Sign-On	Logging in once and being recognised by every connected application